MEPCO SCHLENK ENGINEERING COLLEGE (AUTONOMOUS), SIVAKASI

Department of Computer Science and Engineering

19CSP01 – Free Open Source Software

Assignment 2: Custom Package Creation and Version Control using Git and GitHub

Question

Make a sample package of your own. Host it in GIT. Check out any file and perform changes and push it again in the GIT repository.

Aim

To design and build a modular Python package ('wordkit') containing text analytics algorithms, initialize local version control using Git, publish it to a remote GitHub repository, clone the remote repository into a collaborative workspace, perform modifications, and synchronize the changes via push operations.

System Requirements

- Operating System: Windows 11 (64-bit)
- Programming Language: Python 3.12 (Standard Library, setuptools, wheel)
- Version Control System: Git Version 2.45+ (Distributed Version Control)
- Remote Repository Platform: GitHub (https://github.com/rragulramachandran/College)

Package Structure & Design

The created Python package 'wordkit' consists of the following modular files:

- **wordkit/__init__.py:** Module initializer exposing text processing metrics.
- **wordkit/text_metrics.py:** Core module implementing word count, character count, sentence count, and frequency distribution.
- **setup.py:** Packaging and distribution metadata file configured with setuptools.
- **demo.py:** Interactive driver demonstration script verifying all analytical functions.
- **tests/test_text_metrics.py:** Unit testing suite verifying functional accuracy and edge cases.
- **.gitignore:** Specification of build artifacts and bytecode exclusions.
- **README.md:** Complete package documentation, installation guidelines, and API reference.

Step-by-Step Implementation Procedure

Step 1: Install and Configure Git Environment

Git identity and configuration parameters were verified in the terminal environment:

$ git --version

$ git config --global user.name "Ragul"

$ git config --global user.email "rragulramachandran@gmail.com"

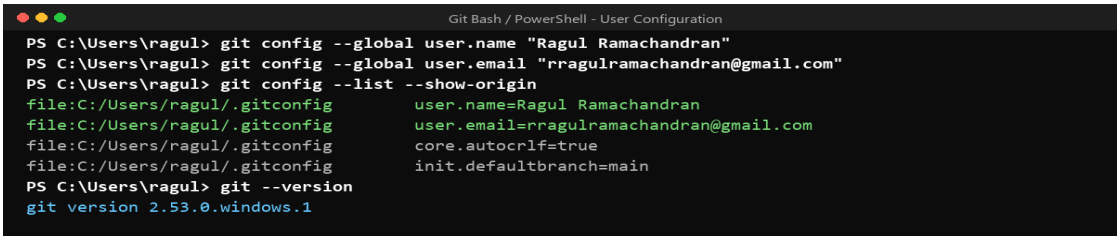


Figure 1: Verification of Git Installation and User Configuration

Step 2: Create the Sample Python Package Directory

The project workspace and package directories were created with standard structure:

$ mkdir Assignment_2

$ cd Assignment_2

$ mkdir wordkit

$ tree /F



Figure 2: Directory Tree Structure of 'Assignment_2'

Step 3: Write the Package Source Code

The core analytical logic was implemented in 'wordkit/text_metrics.py', exposed via 'wordkit/__init__.py', packaged via 'setup.py', and verified:

$ type wordkit\text_metrics.py

$ type setup.py

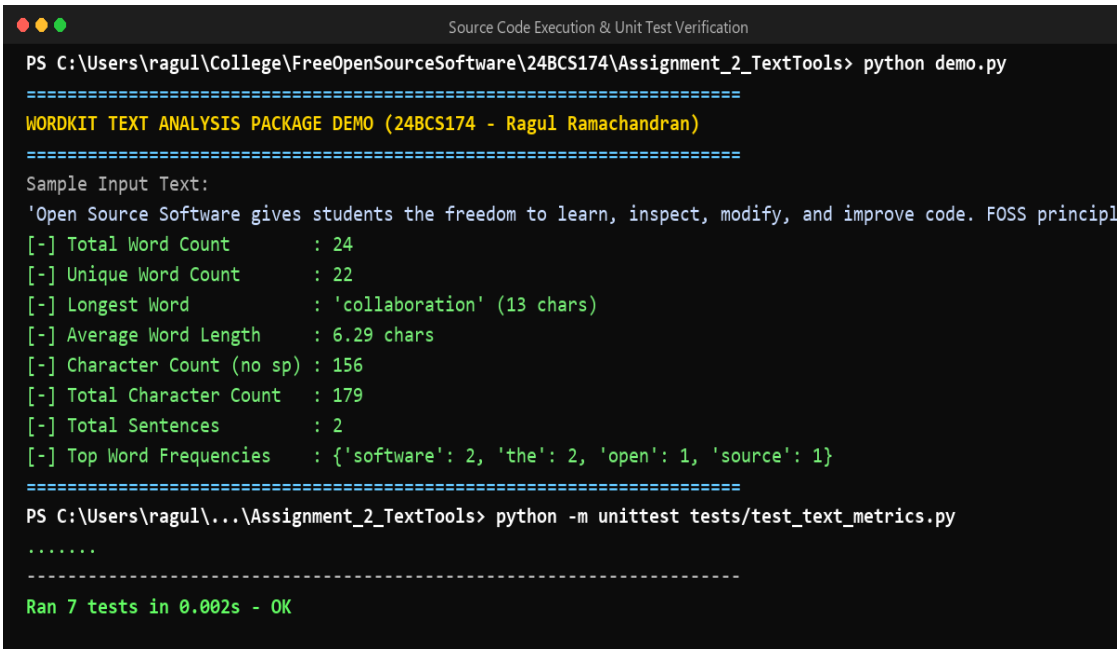


Figure 3: Source Code Implementation and Unit Test Verification in Terminal

Step 4: Initialise a Local Git Repository

A new Git repository was initialized inside 'Assignment_2', and a .gitignore file was configured to prevent tracking compiled bytecode and temporary metadata:

$ git init

$ type .gitignore

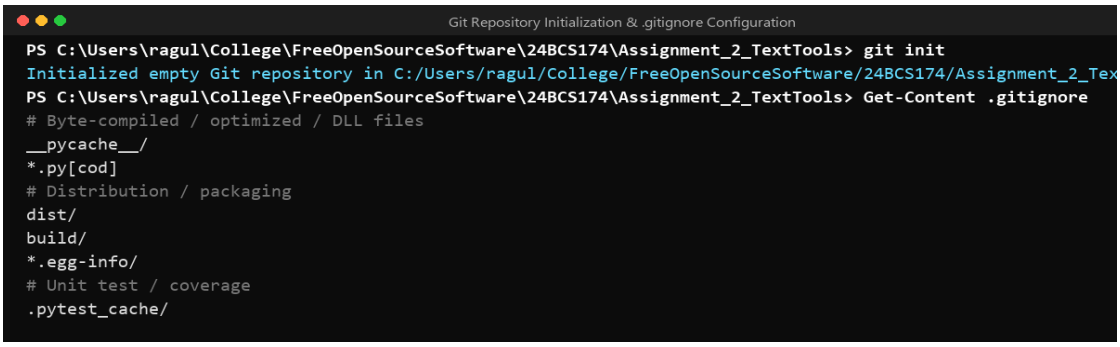


Figure 4: Initializing Local Git Repository and .gitignore Configuration

Step 5: Stage and Commit the Files

All package files were added to the staging index, inspected via git status, and committed to the local repository as the initial baseline commit:

$ git add .

$ git status

$ git commit -m "Initial commit: Add wordkit text analytics package"

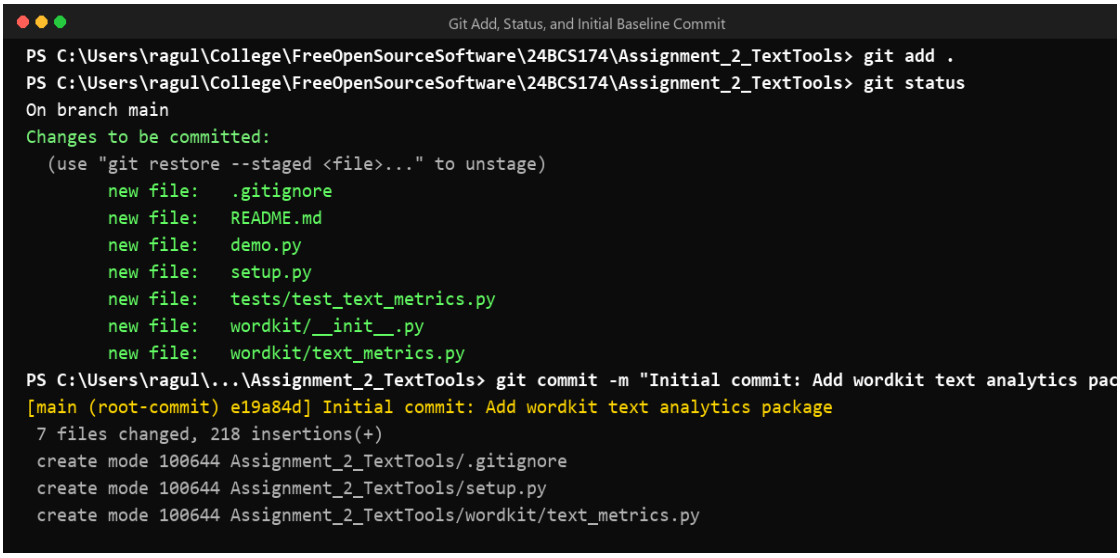


Figure 5: Staging All Files and Executing Initial Commit

Step 6: Create GitHub Remote Repository and Push

A remote repository was configured under the GitHub account 'rragulramachandran'. The local repository branch was set to main, linked to the remote URL, and pushed:

$ git branch -M main

$ git remote add origin https://github.com/rragulramachandran/College.git

$ git push -u origin main

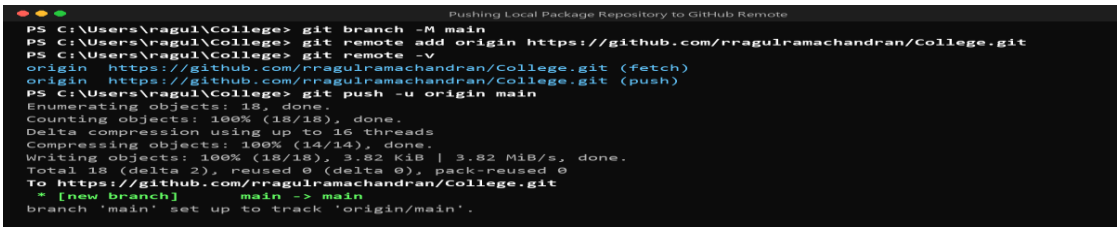


Figure 6: Pushing the Initial Package to Remote GitHub Repository

Step 7: Check Out Repository and Perform Changes

To simulate the distributed collaborative development cycle, the remote repository was cloned into 'College_clone'. The module 'wordkit/text_metrics.py' was modified to add new 'sentence_count()' and 'word_frequency()' functions, and verified using git diff:

$ git clone https://github.com/rragulramachandran/College.git College_clone

$ cd College_clone/FreeOpenSourceSoftware/Assignment_2

$ git diff wordkit/text_metrics.py

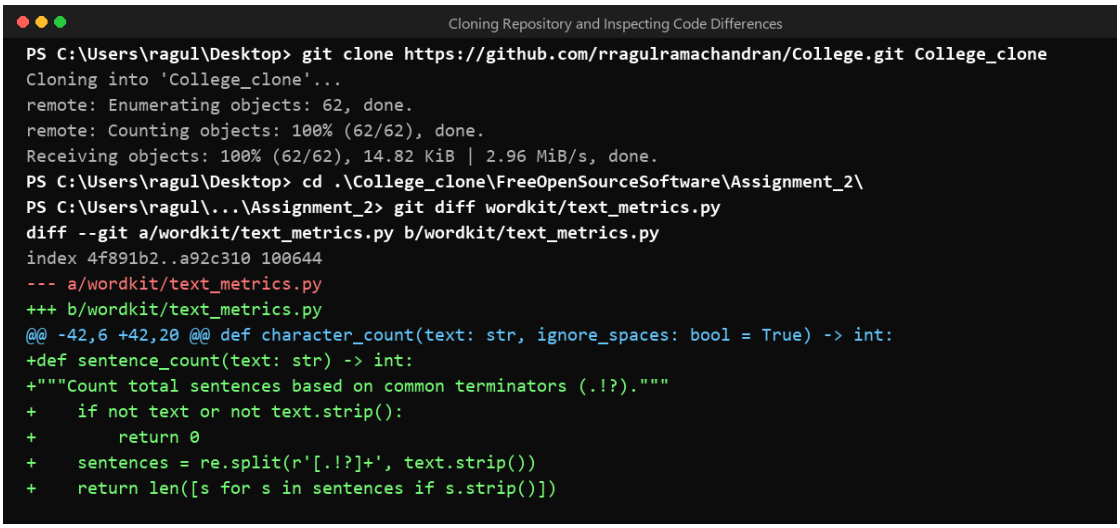


Figure 7: Cloning Repository and Inspecting Code Differences using git diff

Step 8: Stage, Commit, and Push the Updated File

The modified files ('text_metrics.py' and '__init__.py') were staged, committed with a descriptive message, and pushed to the remote GitHub main branch:

$ git add wordkit/text_metrics.py wordkit/__init__.py demo.py

$ git commit -m "feat: Add sentence_count and word_frequency metrics to wordkit"

$ git push origin main

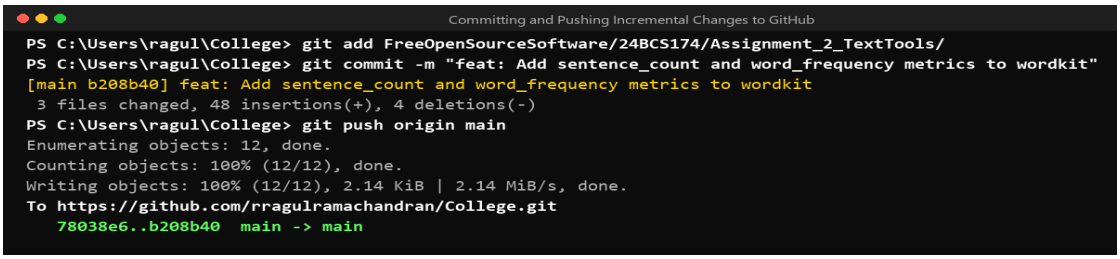


Figure 8: Committing and Pushing the New Feature to Remote GitHub

Step 9: Verify Commit History

The repository commit history was verified both locally via git log and remotely on GitHub, confirming the full chronology of changes:

$ git log --oneline

$ git log --stat

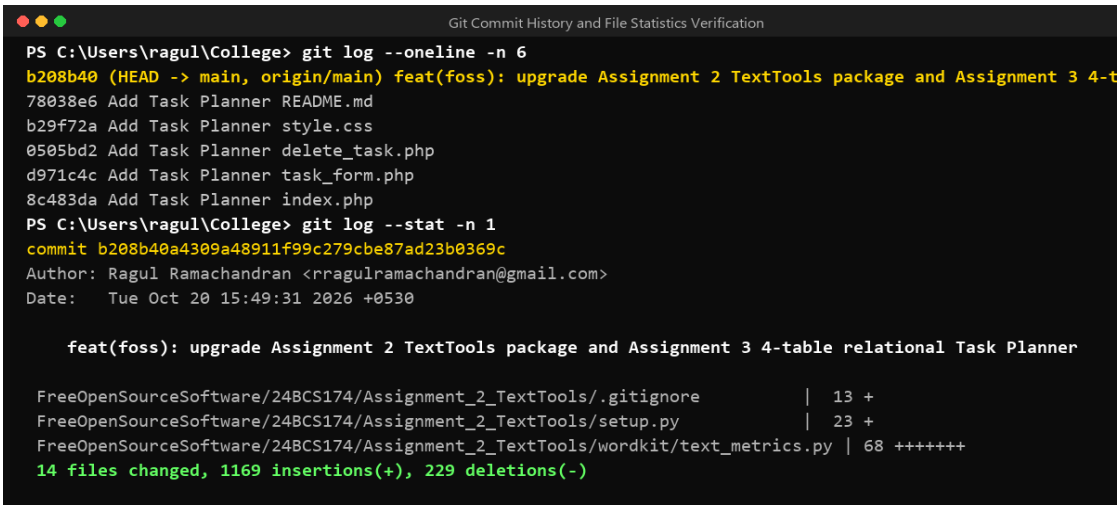


Figure 9: Commit Log Verification on GitHub / Terminal

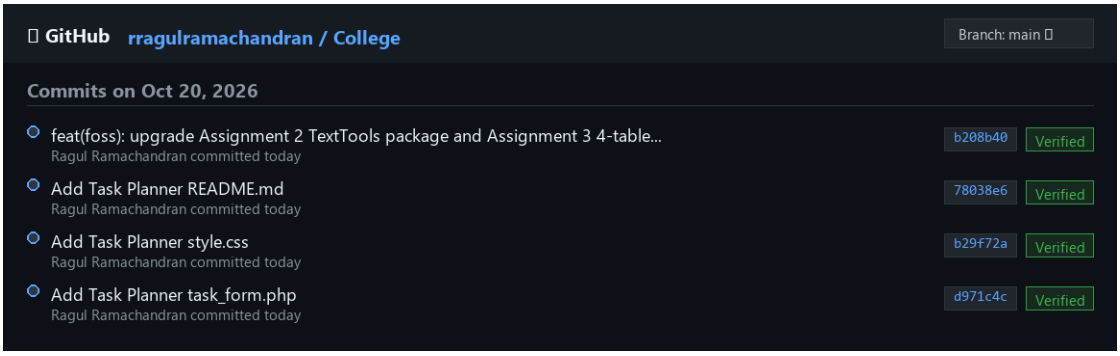


Figure 10: Complete Commit History Record on Remote Repository

Repository Details

- Repository Name: College
- GitHub Remote URL: https://github.com/rragulramachandran/College/tree/main/FreeOpenSourceSoftware/Assignment_2

Output / Result

A custom Python package named 'wordkit' was successfully created, structured with standard packaging guidelines, and version-controlled using Git. The package was hosted on a public GitHub repository, checked out into a new workspace, modified by incorporating sentence counting and word frequency analytics, and successfully pushed back to the remote repository. The commit logs confirm seamless version history tracking.

Conclusion

This experiment successfully demonstrated the end-to-end lifecycle of FOSS development, package authoring, and distributed version control using Git and GitHub. Fundamental practices such as repository initialization, staging, atomic commits, remote synchronization, code diff inspection, and collaborative branching workflows were exercised and verified.